

Handling **Null values, Error values, and Duplicate values** is part of **data cleaning in Exploratory Data Analysis (EDA)**. It improves **data quality, model accuracy, and reliability**.

1. Handling Null Values (Missing Data)

What are Null Values?

Values that are **missing, undefined, or unavailable** in the dataset.

Examples

NaN, NULL, None, blank cells

Why Missing Values Occur

- Data entry errors
 - Data collection issues
 - Sensor failures
 - Merging datasets
 - Optional fields
-

Ways to Handle Null Values

1. Delete Records

Row Deletion

Remove rows containing missing values.

When to use

- Small number of missing records
- Missing values randomly distributed

Pseudo Logic

```
IF row contains null  
  delete row
```

Python

```
df.dropna()
```

Risk

Loss of information.

Column Deletion

Remove columns with too many missing values.

Rule of thumb

If > 40–60% values missing

```
df.drop(columns=['column_name'])
```

2. Imputation (Filling Missing Values)

Mean Imputation (Numerical)

Replace missing values with **mean of column**

```
x_missing = mean(column)
df['age'].fillna(df['age'].mean())
```

Median Imputation

Better for **skewed data or outliers**

```
df['income'].fillna(df['income'].median())
```

Mode Imputation (Categorical)

Fill with most frequent value.

```
df['city'].fillna(df['city'].mode()[0])
```

3. Forward Fill / Backward Fill (Time Series)

Forward Fill

Use previous value
`df.fillna(method='ffill')`

Backward Fill

`df.fillna(method='bfill')`

Used in:

- Stock prices
 - Sensor readings
-

4. Replace with Constant Value

`fillna(0)`
`fillna("Unknown")`

Used when missing has **meaning**.

Example

Children column -> 0 means no children

2. Handling Error Values

What are Error Values?

Incorrect or impossible values.

Examples

Column Error Example

Age	-5
Salary	"ten thousand"
Date	32/13/2023
Gender	"Mle"

Methods to Handle Error Values

1. Validation Rules

Define **logical constraints**

Examples

```
Age >= 0
Salary > 0
Date valid
```

Python Example

```
df = df[df['age'] >= 0]
```

2. Data Type Conversion

Fix incorrect types.

Example

```
"10000" → 10000
df['salary'] = pd.to_numeric(df['salary'], errors='coerce')
```

`errors='coerce'` converts invalid values to **NaN**.

3. Standardization

Correct inconsistent text.

Example

```
Male, male, M → Male  
df['gender'] = df['gender'].str.lower()
```

4. Outlier Detection

Extreme values may indicate errors.

Methods

- IQR
- Z-score
- Box plot

Example

```
Salary = 999999999 → error
```

5. Manual Correction

Domain knowledge correction.

Example

```
Date 20233 → 2023
```

3. Handling Duplicate Values

What are Duplicates?

Same record appearing **multiple times**.

Example

ID	Name	Age
101	John	25
101	John	25

How Duplicates Occur

- Data merging
 - System bugs
 - Data entry repetition
-

Methods to Handle Duplicates

1. Detect Duplicates

```
df.duplicated()
```

Count duplicates

```
df.duplicated().sum()
```

2. Remove Duplicates

```
df.drop_duplicates()
```

3. Remove Based on Specific Columns

Example

Duplicate if **same email**

```
df.drop_duplicates(subset='email')
```

4. Keep First / Last Record

Keep first

```
df.drop_duplicates(keep='first')
```

Keep latest

```
df.drop_duplicates(keep='last')
```

5. Aggregate Duplicates

Example

Multiple sales records

Group by customer and sum purchase

```
df.groupby('customer_id').sum()
```

Summary Table

Problem	Techniques
Null Values	Delete rows, delete columns, mean/median/mode imputation, ML imputation
Error Values	Validation rules, type conversion, standardization, outlier detection
Duplicate Values	Detect duplicates, drop duplicates, subset duplicates, aggregation

Best Practices in Data Science

1. **Never blindly drop data**
 2. **Understand data distribution first**
 3. **Use domain knowledge**
 4. **Check missing patterns**
 5. **Always document cleaning steps**
-

 **Typical Data Science Pipeline**

Raw Data
↓
Missing Value Handling
↓
Error Correction
↓
Duplicate Removal
↓
Outlier Detection
↓
EDA
↓
Feature Engineering
↓
Model Training